

队伍编号	MCB2502562
赛道	(B)

物流理赔风险识别及服务升级问题

摘 要

服务质量与用户体验是物流链中必不可少的一部分。面对增量巨大的物流业务，物品丢失甚至破损的情况会损坏消费者的合法权益，对此进行的理赔服务成为当前物流企业运营的关键环节。本文围绕物流企业理赔服务的风险控制与成本平衡问题，解决三个问题并构建相应模型。

针对问题一，本文先通过先清洗附件 1 中的原始数据，对异常值和缺失值进行处理，并根据“ $\text{索赔差额} = \text{实际赔付金额} - \text{理赔差额}$ ”算出索赔差额。接着用 K-Means、DBSCAN、谱聚类、K-Medoids 等方法对处理后的数据进行聚类分析，建立风险标注模型并对不同方法的结果进行比较。考虑到模型训练中的不平衡数据问题，需要通过 SMOTE、分层聚类、双模型协同等方法对聚类分析的结果进行优化。模型训练完成后，对优化后的模型进行评估，保留效果最优的 DBSCAN+GMM 分层协同聚类分析聚类模型。

针对问题二，本文首先通过特征工程对数据进行处理，避免原始数据带偏预测模型。将附件 1 的数据作为训练数据，附件 2 中的数据作为测试数据。在赔付金额预测过程中，我们采用集成学习法，以 XGBoost、随机森林和 CatBoost 为基模型，最后通过模融合这三个模型来提升预测性能。再用五折交叉验证出每个基模型的预测值，以 Ridge 回归融合三个模型的预测结果并进行最后修正。

针对问题三，问题三要求用问题一中设定的风险标注规则与建立的风险标注模型，预测附件 2 中运单的风险标注结果，还需要描述对严重超额样本占比严重不均衡的办法，并分析当前方法与问题 2 中预测方法的优势与劣势。先要对附件 2 预测后的数据进行处理，算出索赔差额。接着使用问题一中设定的风险标注规则对附件 2 的的运单进行风险标注预测。其中，主要采用 SMOTE 和分层的方法解决严重超额样本严重不均衡的问题。最后将这种直接预测风险标注的方法和先预测实际金额再预测风险标注的方法进行对比，发现先预测实际赔付金额再进行风险分类标注的效果更好。

关键词：聚类模型；机器学习；回归预测；风险识别与预测

目录

- 一、问题重述 1
 - 1.1 背景知识 1
 - 1.2 问题描述 1
- 二、问题分析 1
 - 2.1 问题一分析 1
 - 2.2 问题二分析 2
 - 2.3 问题三分析 2
- 三、模型假设 2
- 四、符号说明 3
- 五、问题一模型的建立与求解 3
 - 5.1 数据预处理 3
 - 5.2 索赔差额计算 5
 - 5.3 DBSCAN+GMM 分层协同聚类分析 5
 - 5.4 风险标注规则与结果 7
 - 5.5 模型评估 8
- 六、问题二模型的建立与求解 12
 - 6.1 数据处理 12
 - 6.2 建立预测模型 14
 - 6.3 预测结果评估与分析 17
- 七、问题三模型的建立与求解 19
 - 7.1 数据处理 20
 - 7.2 直接预测风险标注 20
 - 7.3 两种方法对比分析 25
- 八、模型评价与推广 26
 - 8.1 模型的优点 26
 - 8.2 模型的缺点 27
 - 8.3 模型的改进与推广 27
- 九、参考文献 27
- 十、附录 28

一、问题重述

1.1 背景知识

近年来物流行业快速扩张，服务质量与用户体验是物流链中必不可少的一部分。面对增量巨大的物流业务，物品丢失甚至破损的情况会损坏消费者的合法权益，对此进行的理赔服务成为当前物流企业运营的关键环节。一方面，优质的理赔服务能够更好保障消费者权益，提升客户满意度和品牌形象，增加回头客；另一方面，物流理赔成本关系到企业运营成本，进一步影响企业营业利润。如何在保障客户权益与成本控制之间取得平衡，成为影响物流企业持续健康发展的挑战之一。

物流理赔风险受运单特征、客诉特征、网点特征等多重因素影响。运单特征直接决定赔付基准额，影响客户索赔金额和企业的潜在赔付成本。客户历史的寄件行为、历史理赔情况等客诉特征是评估物流理赔风险的重要参考。不同网点管理水平有差异，高发单量与高赔付率的网点物流理赔风险更高。因此，建立合适的模型来预测需要索赔的订单，既能够减少人工审核压力，也能更好控制理赔风险与成本，对于物流爷也有重要意义。

1.2 问题描述

问题 1：分析附件 1 中“实际赔付金额”和“理赔差额”，根据理赔差额的分布情况设定风险标注规则，建立风险标注模型，分类运单得出风险标注结果并分析。

问题 2：利用附件 1 中的数据建立回归模型，预测附件 2 中运单的实际赔付金额并说明预测结果的评估指标。通过合适的特征选择，回归模型训练、误差评估以及结果预测为运单进行准确的赔付金额预测。

问题 3：用问题 1 中的设定的风险标注规则与建立的风险标注模型，预测附件 2 中运单的风险标注结果。描述对“严重超额”样本占比严重不均衡的处理办法，分析当前方法与问题 2 中预测方法的优势与劣势。

二、问题分析

2.1 问题一分析

问题一要求根据附件 1 数据算出索赔差额，对索赔差额设定风险标注规则，建立风险标注模型，并对运单进行风险分类。要先清洗附件 1 中的原始数据，对异常值和缺失值进行处理，并根据“索赔差额=实际赔付金额-理赔差额”算出索赔差额。接着用 K-Means、DBSCAN、谱聚类、K-Medoids 等方法对处理后的数

据进行聚类分析，建立风险标注模型并对不同方法的结果进行比较。考虑到模型训练中的不平衡数据问题，需要通过 SMOTE、分层聚类、双模型协同等方法对聚类分析的结果进行优化。模型训练完成后，对优化后的模型进行评估，保留效果最优的聚类模型。

2.2 问题二分析

问题二要求结合附件 1 中的数据建立预测模型，预测出附件 2 中每一运单的实际赔付金额并评估，最后填写结果到给定的表格中。首先通过特征工程对数据进行处理，避免原始数据带偏预测模型。将附件 1 的数据作为训练数据，附件 2 中的数据作为测试数据。在赔付金额预测过程中，我们采用集成学习法，以 XGBoost、随机森林和 CatBoost 为基模型，最后通过模融合这三个模型来提升预测性能。再用五折交叉验证出每个基模型的预测值，以 Ridge 回归融合三个模型的预测结果并进行最后修正。

2.3 问题三分析

问题三要求用问题一中设定的风险标注规则与建立的风险标注模型，预测附件 2 中运单的风险标注结果，还需要描述对严重超额样本占比严重不均衡的办法，并分析当前方法与问题 2 中预测方法的优势与劣势。先要对附件 2 预测后的数据进行处理，算出索赔差额。接着使用问题一中设定的风险标注规则对附件 2 的的运单进行风险标注预测。其中，主要采用 SMOTE 和分层的方法解决严重超额样本严重不均衡的问题。最后将这种直接预测风险标注的方法和先预测实际金额再预测风险标注的方法进行对比，分析二者的优劣。

三、模型假设

本文将作如下假设、便于模型的建立与求解：

- 1.假设所给的历史数据均为真实数据。
- 2.假设对于数据中缺失值的处理方式不会对预测结果造成太大误差。
- 3.附件 1 中的历史运单特征（如网点赔付率、线路风险等）在附件 2 的预测时段内保持相对稳定，未因政策、市场或管理变动失效。
- 4.单个客户的多次寄件理赔行为及不同运单之间相互独立，无“恶意索赔”或批次关联等影响模型的关联性行为。
- 5.假设附件 2 中待预测运单的业务场景（如运输线路类型、商品品类占比）与附件 1 训练集的业务场景分布一致，无新增未在训练集中出现的特殊业务类型。
- 6.假设“实际赔付金额”“索赔金额”等核心变量的统计分布特征，可代表该物流企业同类运单的普遍分布情况，无特殊业务场景（如特定节日大促、临时专线运输）导致的数据偏态。

四、符号说明

符号	说明
A_{actual}	实际赔付金额
A_{claim}	索赔金额
Δ	索赔差额
C_{dense}	DBSCAN 模型的潜在合理诉求
X_{noise}	DBSCAN 模型的潜在异常点
K	分类模型簇数
μ_i	第 i 个簇的质心
y_i	实际值
$\hat{y}_i$	预测值
w_n	第 n 个参数的权重

五、问题一模型的建立与求解

5.1 数据预处理

通过对数据的查看，可以发现部分数据存在异常，且有些数据缺失，此类情况不仅可能影响后续的分析过程，还可能影响最终结果的准确性。为确保数据质量并提升分析准确性和可靠性，要对数据进行预处理。

（1）重复值的处理

通过附件 1 数据的筛选，发现并处理了 25 条重复值，更好保障数据分析的准确性和机器学习模型的可靠性。

（2）异常数据的修正

通过分析附件 1 与附件 2 的数据，本文删除了寄件人、收件人 ID、发出城市、目的城市、寄件是否内部这五个变量。“寄件人 ID”与“收件人 ID”变量与其他变量之间关系较弱，应当直接删除。对比发现城市发单量、赔付率和赔付比率三个变量在不同城市之间有独特性，在机器学习中足以捕捉识别各城市特征，故删除了“发出城市”和“目的城市”两个变量。“寄件是否内部”变量的取值分布 90%为-1，与运单相关特征信息解释中的信息相悖，且问题数据占比超过 90%。为了保证模型的稳健性与准确性，本文认为此项数据存在问题，选择直接删除。

（3）缺失数据的处理

通过对附件 1 和附件 2 数据的筛选，发现缺失值主要存在于“异常原因”和“进线渠道”两列（如图 1 所示），且数量较大，不宜直接删除。本文用 Fillna

方法将两列的缺失值进行填充,进行独热编码后与原数据合并,并删除原始数据。

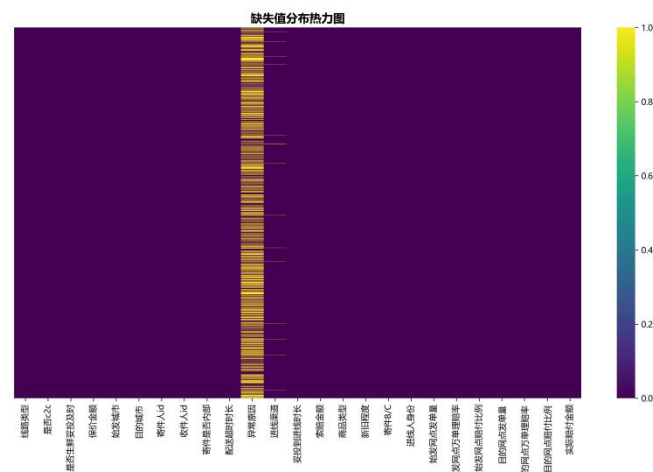


图 1 缺失值处理

(4) 描述性统计

在数据处理后,我们对余下的 11142 条数据进行描述性统计。关键数值变量的统计特征如下图所示,索赔金额与实际支付金额相关性较强,这些统计特征为后续建模提供了重要的数据基础。

表 1 关键变量的描述性统计

	<i>mean</i>	<i>std</i>	<i>min</i>	25%	50%	75%	<i>max</i>
运单保价金额	207.611	310.220	-1.000	14.713	83.813	266.613	3212.917
索赔金额	807.359	805.090	100.000	239.933	545.055	1063.498	5498.770
实际赔付金额	292.825	281.826	0.740	89.690	200.880	409.635	3171.470
始发网点事故率	30.712	38.110	-1.000	4.178	17.118	42.916	323.714
始发网点实际理赔比例	0.968	0.046	0.336	0.961	0.983	0.994	1.000
目的网点事故率	25.852	33.448	-1.000	2.906	13.584	36.267	269.666
目的网点实际理赔比例	0.964	0.041	0.628	0.950	0.977	0.992	1.000
索赔差额	-514.534	617.982	-4463.580	-689.285	-281.865	-107.380	-0.080

(5) Z-Score 标准化

Z-Score 标准化,也成为标准差标准化或零均值标准化,其核心目标是将原始数据变换为均值为 0,标准差为 1 的数据集。这种方法通过衡量每一个数据点距离平均值有多少个标准差来消除数据的不同量纲和自身变异大小的影响,使得不同特征或不同数据集之间的比较变得有意义。模型表达为:

$$z_i = \frac{x_i - \bar{x}}{s}$$

其中为 $\bar{x}$ 特征均值, s 为标准差。标准化后,每个特征的均值为 0、方差为 1,保证聚类时各指标贡献均衡。

本文通过 Z-Score 标准化,将不同尺度和分布的数据转换为标准正态分布,极大地提升了模型的性能和稳定性。

5.2 索赔差额计算

在物流理赔过程中，计算“索赔差额”是分析和标注风险的首要步骤。索赔差额的计算基于客户提出的赔偿金额和物流公司实际支付的赔偿金额之间的差异，不仅反映了赔偿金额的合理性，也为后续的风险分类提供了基础数据。公式表示为：

$$\Delta = A_{\text{actual}} - A_{\text{claim}}$$

其中， Δ 表示“索赔差额”， A_{actual} 为物流公司支付的“实际赔付金额”， A_{claim} 为客户要求的“索赔金额”。理赔过程中，物流公司需计算每个运单的“索赔金额”，该差额用于评估客户的赔偿请求是否合理。

索赔差额是衡量实际赔偿与客户要求之间的偏差。对于一个运单，如果索赔差额为零，则意味着客户的赔偿要求和实际赔偿金额完全一致，属于“合理诉求”；如果理赔差额为负且较大，则意味着客户的索赔要求偏高或超出了合理范围，属于“诉求偏高”或“严重超额”。经分析发现“索赔差额” ≤ 0 ，

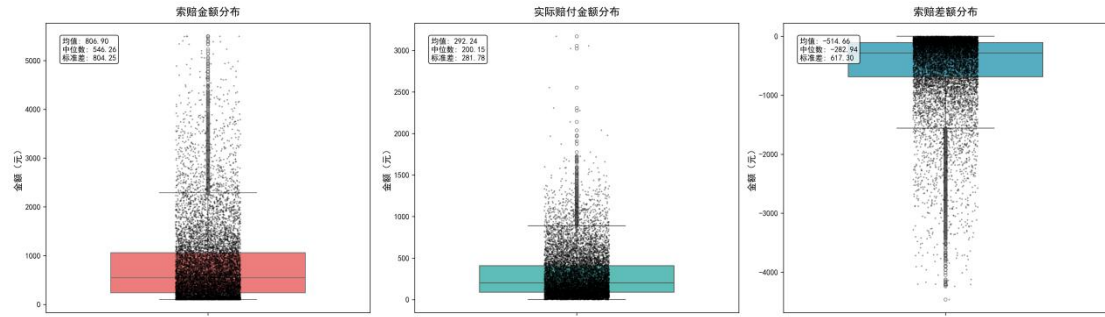


图 2 索赔金额、实际赔付金额与索赔差额分布图

5.3 DBSCAN+GMM 分层协同聚类分析

5.3.1 聚类原理

理赔差额计算后需要对数据进行分类，为每个运单根据理赔差额进行风险标注，需要采用聚类模型对“实际赔付金额”和“索赔金额”进行聚类。DBSCAN 无需预设簇数，且能自动识别“密集簇”（样本集中区域）和“噪声点”（孤立样本），恰好与“合理诉求样本密集，严重超额样本系数”的业务数据特征，同时运用分层 GMM 高斯混合模型，弥补 DBSCAN 无概率输出的缺陷，避免漏判边缘样本，增强结果的可靠性。

该模型首先需要用分层 DBSCAN 密度聚类对每个金融曾 L 独立运行算法，初步分离密集簇 C_{dense} （潜在合理诉求）和噪声点 X_{noise} （潜在异常点）。此阶段后 GMM 只在 C_{dense} 上训练，从而避免噪声点干扰。

对 DBSCAN 找出的密集簇进行软划分，在每一层的密集簇数据 C_{dense} 上训练高斯混合模型。簇数 K 固定为 3，其概率密度函数为：

$$p(x|\theta) = \sum_{k=1}^3 \pi_k \cdot N(x|\mu_k, \Sigma_k)$$

接着使用 EM 算法在 X_{clean} 上训练 GMM，计算每个点 x_i 属于各分类 k 的后验概率 γ_{ik} ，表示点 x_i 属于簇 k 的概率，再使用 E 步计算出的 γ_{ik} 来更新模型参数 θ ，以最大化对数似然函数，重复 E 步和 M 步，直到参数收敛。

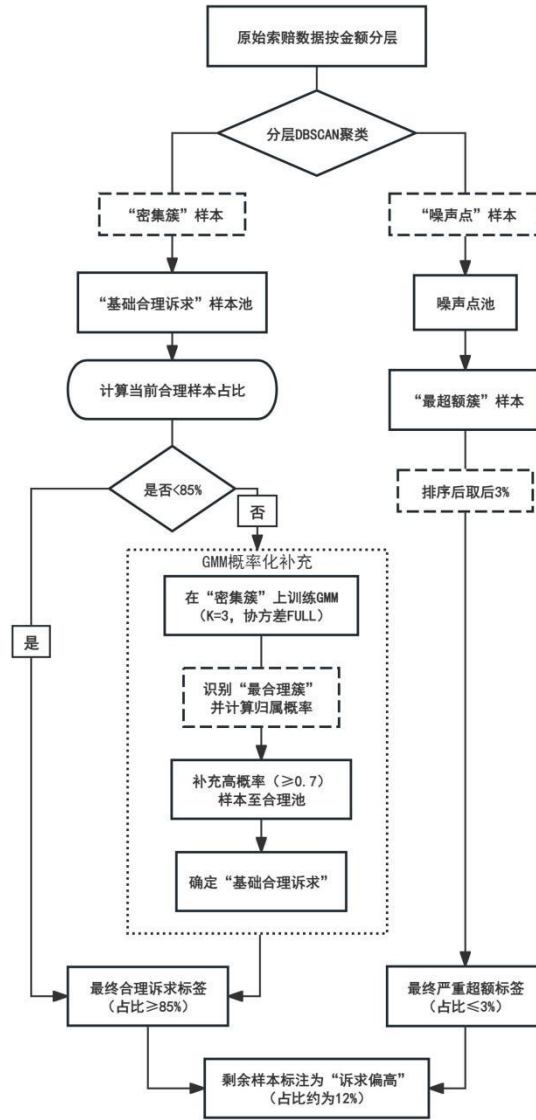


图 3 DBSCAN+GMM 分层协同聚类思路图

最后为所有点（包括 C_{dense} 和 X_{noise} ）赋予最终标签，实现最终聚类决策，得出最终聚类与概率归属结果。

5.3.2 模型应用步骤

第一，分层 DBSCAN 密度聚类。针对不同金融层的差额分布差异，设置动态领域半径（ eps ），金额越高， eps 越大，就能够容忍更宽的合理差额。本文将各层最小样本数（ $min_samples$ ）统一设为 5，确保密集簇识别的稳定性。

表 2 动态领域半径设置

层数	金额	<i>eps</i>
1	低额	0.3
2	中低额	0.5
3	中高额	0.7
4	高额	0.9

第二，使用分层 GMM 高斯混合模型进行概率化验证与簇校准

按照“索赔差额_z-score 标准化均值”对 GMM 输出的 3 个簇进行升序排序，得出三类：最合理簇（均值最小）、中等簇、最超额簇（均值最大）。接着通过概率筛选输出样本属于“最合理簇”的概率，仅保留概率大于 0.7 的高置信度样本补充至“合理诉求候选池”，避免低置信度边缘样本误判。其中，簇数固定为 3，协方差类型设为“FULL”以适配高二层复杂的数据分布，初始化次数设为 10 以避免局部最优解。

第三，风险标注。以 DBSCAN 密集簇为基础，以 GMM 概率补充，最终按业务占比约束锁定标签，确保结果满足“合理诉求 $\geq 85\%$ ”和“超额诉求 $\leq 3\%$ ”的要求。

5.4 风险标注规则与结果

基于 DBSCAN+GMM 协同聚类模型，本文设定如下风险标注规则并建立风险标注模型：

（1）合理诉求标注（各层最终合理诉求占比均为 85.02%-85.03%，完全达标）：

1. 优先选取 DBSCAN 密集簇样本即基础合理样本
2. 若密集簇样本量不足 85%，则需要从“最合理簇”中补充高概率（如 ≥ 0.7 ）样本，直至满足“每层合理诉求占比 $\geq 85\%$ ”的要求。

（2）严重超额标注（各层严重超额占比均为 2.97%，符合约束）：

1. 候选池为 DBSCAN 噪声点与 GMM “最超额簇”样本
2. 按“索赔差额_z-score 标准化降序”排序，选取后 3%样本，确保占比 $< 3\%$

（3）诉求偏高标注（各层占比均为 12%，符合要求）：

介于合理与超额之间剩余未被标注的样本统一标注为“诉求偏高”

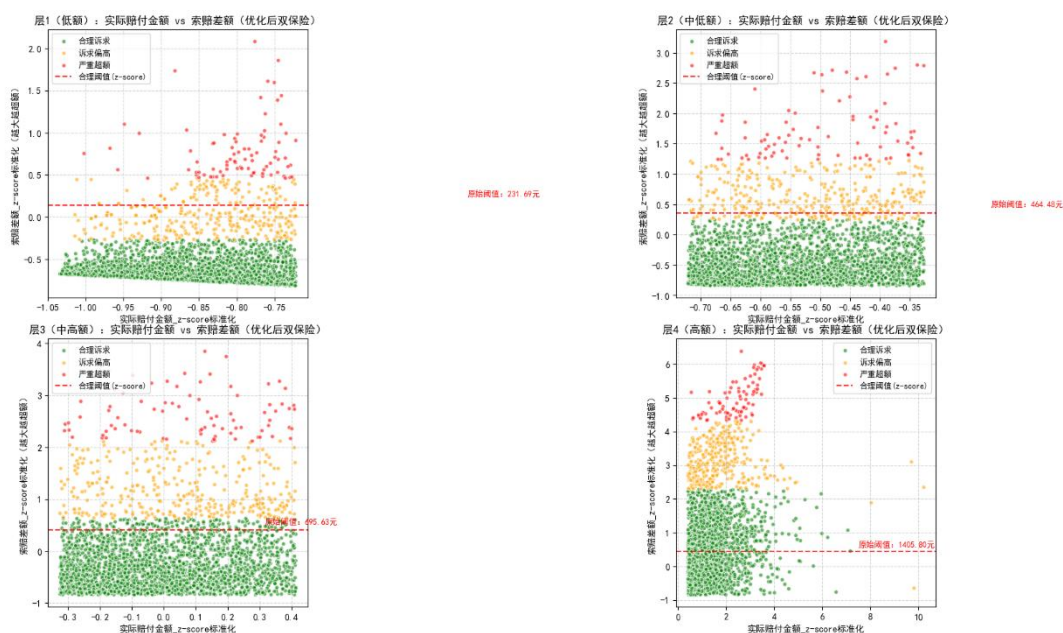


图 4 聚类结果散点图

基于分层聚类与密度分析相结合的风险标注方法，对附件 1 中的 11,142 条运单数据进行风险标注，得到如下分类结果：

表 3 分层聚类结果

风险类别	个数	占比（%）
合理诉求	9474	85.03
诉求偏高	1337	12.00
严重超额	331	2.97

结果显示，各层最终合理诉求类别占比均为 85.02%-85.03%，完全达标；诉求偏高类别各层占比均为 12%，符合要求；严重超额类别各层严重超额占比均为 2.97%，符合约束。本风险标注模型成功实现了对附件 1 运单数据的准确分类，各项占比指标均严格满足业务约束条件。

5.5 模型评估

本文通过轮廓系数、Calinski-Harabasz 指数、Davies-Bouldin 指数等无监督聚类来量化聚类质量：

1、轮廓系数：对于一个样本集合，它的轮廓系数是所有样本轮廓系数的平均值。轮廓系数的取值范围是[-1,1]，同类别样本距离越相近不同类别样本距离越远，分数越高，聚类效果越好。

2、DBI（Davies-bouldin）：该指标用来衡量任意两个簇的簇内距离之后与簇间距离之比。该指标越小表示聚类效果越好。

3、CH（Calinski-Harabasz Score）：通过计算类内各点与类中心的距离平方

和来度量类内的紧密度（分母），通过计算类间中心点与数据集中心点距离平方和来度量数据集的分离度（分子），CH 指标由分离度与紧密度的比值得到，CH 越大表示聚类效果越好。

5.5.1 DBSCAN+GMM 分层协同聚类模型评价

本文对分层 DBSCAN+GMM 双保险聚类方法的聚类性能进行量化评估，结果如下：

表 4 表 4 DBSCAN+GMM 双保险聚类模型评估		
轮廓系数	Calinski-Harabasz	Davies-Bouldin
0.7489	7590.76	0.8626

首先，模型轮廓系数（Silhouette Score）为 0.7489，该指标衡量簇内样本的紧凑度与簇间样本的分离度，取值范围为[-1,1]。本次实验中轮廓系数达 0.7489，表现非常优秀，说明样本在所属簇内的相似度极高，且与其他簇的差异显著，聚类结构的区分度与紧凑性表现优异。其次，本模型的 Calinski-Harabasz（CH）指数为 1590.76，显著高于一般聚类方法的平均水平，证明分层 DBSCAN 对簇的划分具备极强的“分离性”与“紧凑性”，聚类逻辑与数据分布高度契合。最后，本模型的 Davies-Bouldin（DB）指数为 0.9626，处于“优秀”区间，说明各簇间的独特性明显，不存在簇间混淆或重叠的问题，聚类结果的可解释性与区分度极佳。

5.5.2 其他模型的结果与评估

除 DBSCAN+GMM 分层协同聚类模型之外，本文还尝试了 K-Means 分层聚类、分层动态阈值谱聚类、K-Medoids 聚类等方法进行风险标注。

（1）分层动态阈值谱聚类

分层动态阈值谱聚类是一种结合了分层聚类思想、动态阈值技术和谱聚类优势的改进算法，能够处理复杂数据，优化聚类质量，提高稳定性，因此本文在尝试过程中使用这种方法进行聚类。在构建特征工程后对金额分成四层——按 25% 分位数将金额分为 L1（低）-L4（高）四层，确保同层内金额相近。之后使用分层隔离的方法优化聚类，按金额层独立执行谱聚类，避免高金额样本的宽阈值干扰低金额样本的严格判断，并且采用动态参数分层调整谱聚类的 n_neighbors（近邻数）——L1=8、L2=10、L3=12、L4=15，金额越高近邻数越大，允许更宽的差额分布。接着用核密度估计，确保“合理诉求”对应最密集的簇、“严重超额”对应最稀疏的簇，严格匹配题目要求。

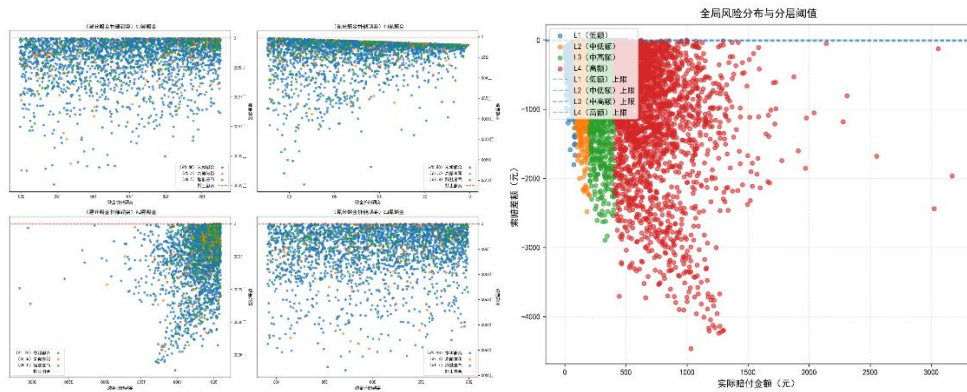


图 5 分层动态阈值谱聚类聚类散点图

最终结果如下，和题目要求有所偏离。

表 5 分层动态阈值谱聚类聚类结果

分层动态阈值谱聚类风险标注结果		
风险类别	个数	占比 (%)
合理诉求	9815	88.09
诉求偏高	448	4.02
严重超额	879	7.89

轮廓系数（Silhouette Score）为 0.0067，该值接近 0，说明聚类结果中样本与自身簇的相似度和与邻近簇的相似度差异较小，聚类边界不够清晰，簇内样本的一致性较低；Calinski-Harabasz 指数为 1.3818 该指数数值较小，表明聚类的分离度较低（簇间差异小、簇内差异相对较大），整体聚类结构的稳定性较弱；Davies-Bouldin 指数：35.7131 该指数值较大（理论上越接近 0 越好），说明簇与簇之间的相似度较高，聚类的区分度不佳。

综合来看，当前聚类结果的质量整体偏低，簇内样本的一致性和簇间的差异性不够理想，因此团队在经过一系列优化后没有选择谱聚类模型。

（2）K-Means 分层聚类

K-Means 的核心目标是将给定的数据集划分成 K 个簇(K 是超参)，并给出每个样本数据对应的中心点。在本题中，可以直接将样本划分为指定的 3 簇（即合理诉求、诉求偏高与严重超额）并且对“索赔差额”这类连续型特征的聚类效果稳定。因此本文首先运用分层独立聚类的方法，在对数据进行 Z-Score 标准化处理后，对数据进行层次划分，以“实际赔付金额”的 25%、50%、75%分位数为阈值，将全量数据划分为 4 个金额层，确保 q 每层内运单金额特征相近，层间差异显著，并且建立分层函数，每层单独执行 K-Means 聚类，簇数为 3 并使用多次初始化聚类中心，多次迭代、固定随机种子的方法进行稳定性优化，对每层金额相近的样本单独执行 K-Means（K=3），避免金额量级差异干扰聚类结果。

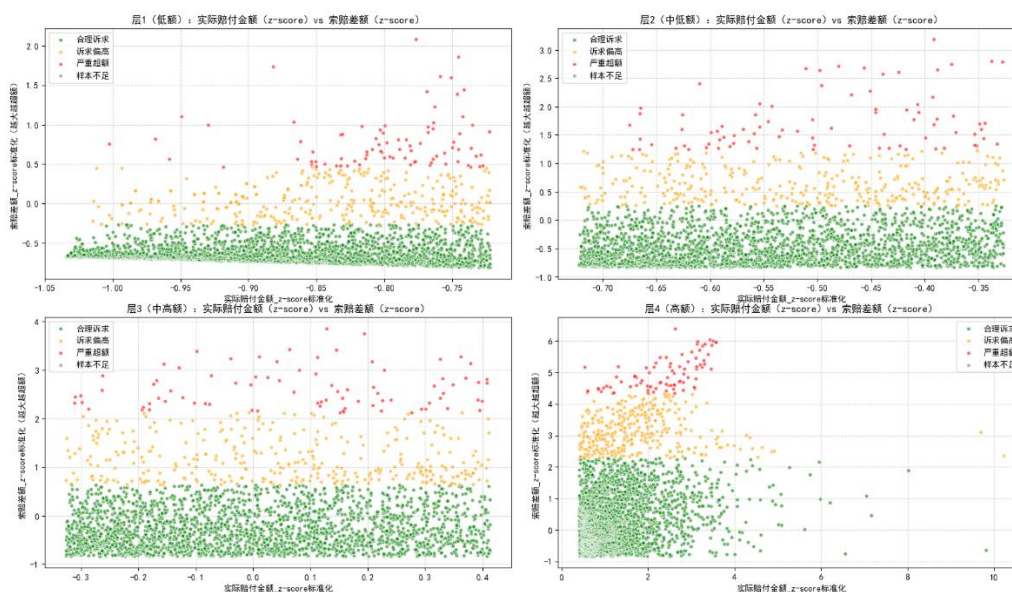


图 6 K-Means 分层聚类散点图

如图所示，合理诉求占比 85.3% ($\geq 85\%$)，诉求偏高占比 12%，严重超额占比 2.97% ($\leq 3\%$)，风险标注结果符合要求，需要对模型进行进一步评估。

如图所示，该方法下各层轮廓系数均在 0.47 以上，CH 指数均在 2400 以上，说明分层聚类的簇内紧凑型，簇间分离度较好，结果可靠。

表 6 K-Means 分层聚类评估

层级	有效样本数	轮廓系数	Calinski-Harabasz	Davies-Bouldin
层 1 (低额)	2792	0.6071	5673.17	0.6109
层 2 (中低额)	2792	0.5712	5934.44	0.6023
层 3 (中高)	2791	0.5190	5461.26	0.6532
层 4 (高额)	2792	0.4744	2494.19	0.8896

(3) K-Medoids 聚类

K-Medoids (K-中心点) 是 K-Means 的一种改进算法，将数据集中的 n 个对象划分为 k 个簇，使得每个对象与其所属簇的中心点之间的距离之和最小。本文在选择模型过程中尝试 K-Medoids 聚类方法，以实际数据为中心点，实际赔付金额和索赔差额作为聚类特征，调整簇边界确保"严重超额" $< 3\%$ ，"合理诉求" $\geq 85\%$ ，控制金额敏感性，使高赔付金额需要更大的索赔差额被标记为高风险，并且确保不同类别的索赔差额密集程度有明显差别，并且 K-Medoids 支持任意距离度量（只要能计算样本间的距离矩阵），可灵活集成业务规则定义的距离，在保证聚类稳定性的同时，提升结果的业务可解释性，便于风险标注的落地与复核。



图 7 K-Medoids 聚类散点图

如表所示，结果较为符合题目要求，于是本文继续使用无监督学习模型评估指标对该模型进行评估。。

表 7 K-Medoids 聚类评估结果

风险标注结果		
风险类别	个数	占比 (%)
合理诉求	9530	85.53
诉求偏高	1334	11.97
严重超额	278	2.5

评估结果（需要一个表）显示，轮廓系数为 0.4168 属于中等偏上水平，表明大部分样本能较好地归属于所在簇。Calinski-Harabasz 指数 9553.18 属于极高水平，远大于常规有效聚类的阈值说明簇与簇之间的差异非常显著，且每个簇内部的样本分布高度集中。Davies-Bouldin 指数 0.9705，属于优秀水平，表明簇内样本的分散程度低。综合来看，该模型聚类效果较好，能够有效挖掘数据的内在结构，为最终的模型选择提供很大的参考价值。

六、问题二模型的建立与求解

6.1 数据处理

6.1.1 正态性检验

为验证使用变量的前提条件，本文对预处理后的“实际赔付金额”与“索赔差额”两个变量进行正态性检验。从图 8 可以发现“实际赔付金额”和“索赔差额”明显偏离正态分布，呈现出严重的右偏分布，数据集中在较小值，右侧存在长尾。

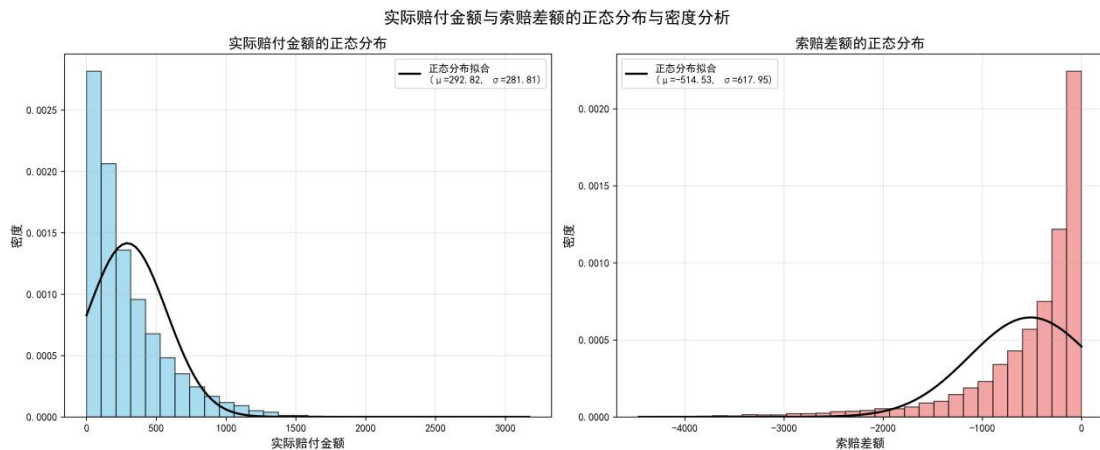


图 8 实际赔付金额与索赔差额的密度分布

从 Q-Q 图可看出蓝色散点在中间部分与红线拟合较好，右侧尾部的散点明显高出了红色对角线，显示出右偏分布。这说明，数据中存在少数赔付金额非常高的运单，如果直接用于机器学习会影响预测模型准确性。

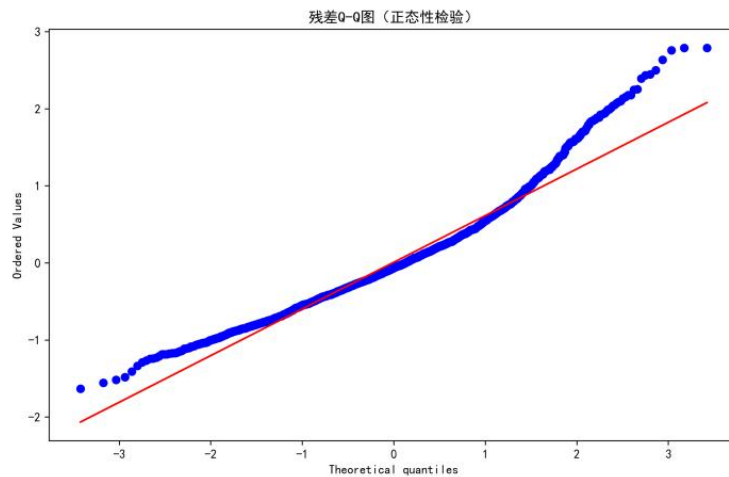


图 9 实际赔付金额与索赔差额的残差 Q-Q 图

6.1.2 特征工程

特征工程是指利用数据领域的相关知识，对原始数据进行处理、转换、组合，构建出更适合机器学习模型使用的特征变量。附件 1 与附件 2 中的数据存在原始问题：数值量纲差异大，“保价金额”，“发单量”等变量数额大，易带偏模型；类别特征不能直接使用，如“线路类型”“商品类型”等；变量之间存在潜在的非线性关系，需要进行组合变换。

本文通过交叉特征新生成“始发网点风险”“目的网点风险”两个变量。由于始发网点和目的网点的特征对于实际赔付金额存在影响，因此新增这两个变量用于更好捕捉数据特征。

$$\text{网点风险} = \text{赔付率} \times \text{赔付比例}$$

本文对“实际赔付金额”取对数处理，对其余连续变量进行标准化处理。通

过画出各个变量的散点图，发现大部分变量明显存在长尾分布的特点，直接进行机器学习的效果不佳。部分变量存在负值，无法进行对数化处理，每个变量最小平移距离不同也使得平移取对数较为复杂。为使异常值更加稳健并保留数据的分布特征，本文对这些变量统一使用 Z-Score 标准化处理，这种处理方法可以在消除量纲的同时保持相对差异。

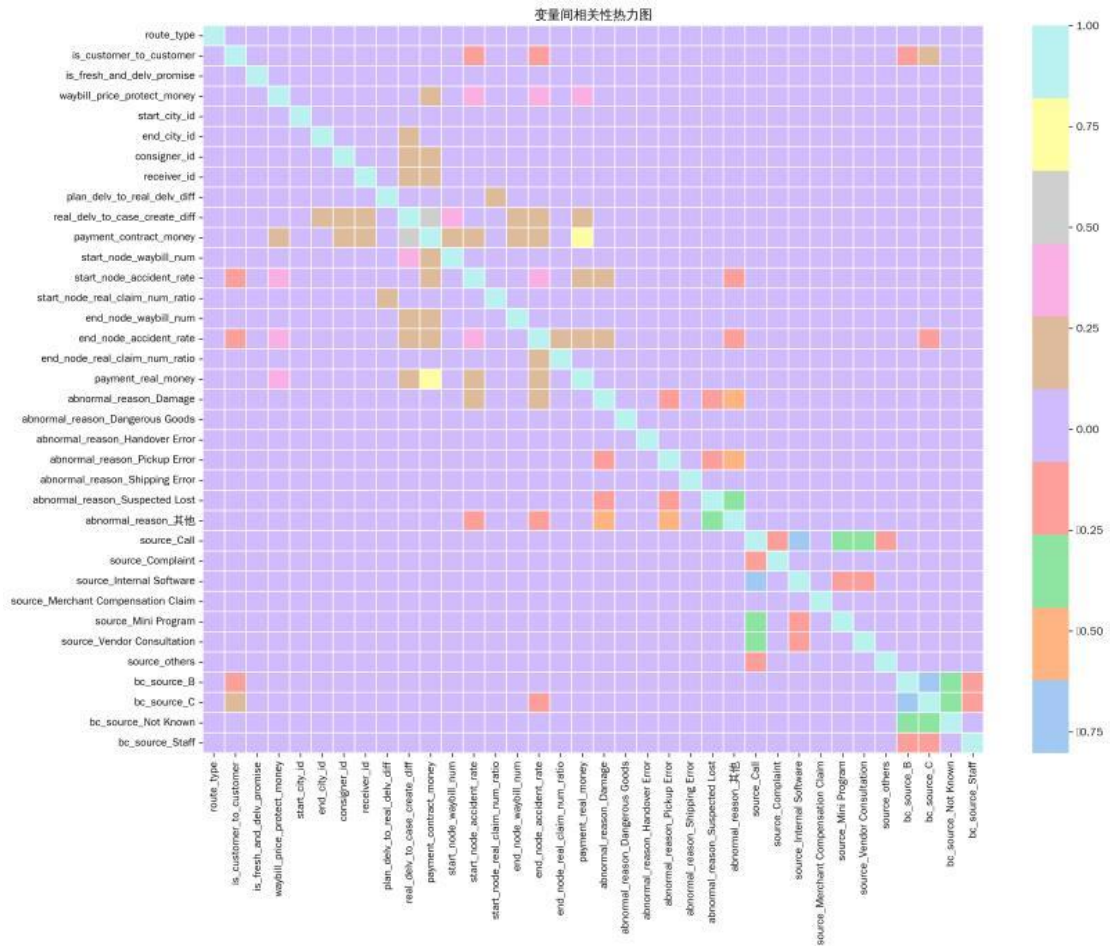


图 10 变量间相关性热力图

本文还对类别变量进行了 One-Hot 编码处理。在进行相关性分析之后，发现类别变量与实际赔付金额之间存在非常显著的相关性，需要将其纳入模型之中分析。考虑到不同类别变量存在多种文字形式，本文先将不同文本类别转化为数字类别，再运用 One-Hot 编码对数字类别进行处理，得到可纳入模型的变量形式。

6.2 建立预测模型

为提高预测结果准确性，避免单模型数据效果不佳，本文采用集成学习法，以 XGBoost、随机森林和 CatBoost 三个模型为基模型进行训练和预测。得出结果后，用五折交叉验证每个基模型的预测值是否符合要求，再用岭回归加权融合三个模型的预测结果并进行最后修正。

在训练三个基模型的过程中，本文通过贝叶斯优化寻找最优参数。贝叶斯优

化可以同时优化连续参数，整数参数，类别参数等不同类型的参数，能够更好提升预测模型的准确性。

6.2.1 XGBoost 算法

XGBoost 是一个基于梯度提升树（GBDT）的机器学习算法，通过优化目标函数来训练模型，目标函数包含损失函数和正则化项两部分。损失函数衡量模型预测与实际标签之间的差异，正则化项则控制模型复杂度，避免过拟合。

$$\mathcal{L}(\theta) = \sum_{i=1}^n \ell(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

其中， $\mathcal{L}(\theta)$ 为 XGBoost 的目标函数， $\ell(y_i, \hat{y}_i)$ 为损失函数，表示第 i 个样本的实际值和预测值之间的差异； $\Omega(f_k)$ 为正则化项，表示树模型的复杂度。

损失函数是衡量模型预测值与实际标签之间差异的指标。在 XGBoost 中，通常是用均方误差（MSE）作为损失函数，公式为：

$$\ell(y_i, \hat{y}_i) = \frac{1}{2} (y_i - \hat{y}_i)^2$$

损失函数通过计算实际标签 y_i 与预测值 $\hat{y}_i$ 之间的差异来衡量模型的表现。在回归问题中，常用的损失函数是均方误差（MSE），它能够量化模型预测值与实际值之间的平方差异。XGBoost 通过最小化损失函数来优化模型的预测能力。每次迭代中，模型通过计算损失函数的梯度来更新参数，减少预测误差。

正则化项用于控制模型的复杂度，防止过拟合。XGBoost 的正则化项包括两个部分：树的复杂度（树的叶子数）和叶子节点的权重大小。具体的正则化表达式为：

$$\Omega(f_k) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

γT 是树的复杂度项，其中 T 是树的叶子节点数量， γ 是控制树复杂度的超参数。增加叶子节点数会增加模型的复杂度，可能导致过拟合，因此需要通过正则化来控制叶子节点的数量。 $\frac{1}{2} \lambda \sum_{j=1}^T w_j^2$ 是对叶子节点权重的正则化项，其中 w_j 是第 j 个叶子节点的权重， λ 是正则化参数。这个项通过惩罚较大的叶子节点权重来减少模型的复杂度。

6.2.2 随机森林算法

随机森林是基于决策树概念提出的一种不相关多棵决策树的集成学习方法，可以在一定程度上消除决策树的缺点。随机森林能够处理复杂的非线性关系，不容易过拟合，自动处理特征之间的交互作用且不需要预设特征之间的关系。

步骤如下：

Step1：有放回地从原始数据集中随机抽样，每次选取 n 个样本量作为一个

训练集；

Step2: 对所抽样样本，随机不重复地抽取 d 个特征作为训练决策树的输入，利用 d 个特征分别对样本集进行划分，构建决策树；

Step3: 重复步骤 1-2 共 k 次，生成 K 棵决策树，构成随机森林；

Step4: 综合 K 棵决策树的分类结果，得到最终分类结果。

特征重要性评估：

通过量化每个特征对构建的 K 棵决策树分类性能的贡献度来对特征进行排序，贡献度常用袋外错误率作为评价指标。

定义指示函数：

$$I(x, y) = \begin{cases} 1, & x = y \\ 0, & x \neq y \end{cases}$$

第 m 个特征值 F_M 在第 k 棵树的 $FIM_{km}^{(OOB)}$ ：

$$FIM_{km}^{(OOB)} = \frac{\sum_{p=1}^{n_0^k} I(Y_p, Y_p^K)}{n_0^k} - \frac{\sum_{p=1}^{n_0^k} I(Y_p, Y_{p,\pi_m}^K)}{n_0^k}$$

式中： n_0^k 为第 k 棵树观测样本数， Y_p 为第 p 个样本对应的真实分类标签， Y_p^K 伪随机置换 F_M 前第 k 棵树对袋外数据第 p 个观测的预测分类结果， Y_{p,π_m}^K 为随机置换 F_M 后第 k 棵决策树对第 p 个样本的分类结果，其中随机置换 F_M 后第 k 棵决策树需要重新训练。当特征值 F_M 没有在第 k 棵树出现时，则 $FIM_{km}^{(OOB)}=0$ 。

特征 F_M 在整个随机森林中的重要性评分定义为

$$FIM_{km}^{(OOB)} = \frac{\sum_{k=1}^K FIM_{km}^{(OOB)}}{K_\sigma}$$

式中： K 表示随机森林中决策树的个数， σ 表示 $FIM_{km}^{(OOB)}$ 的标准差，特征 F_M 的重要评分 $FIM_{km}^{(OOB)}$ 表示 F_M 对分类正确率的贡献度，特征重要性评分是由袋外错误率均值和标准差共同决定的。

（2）模型预测结果与评估

通过上述随机森林模型对附件 2 的实际理赔金额进行预测，从结果可以看出该模型能够较好模拟

表 8 随机森林模型评估

	MSE	$RMSE$	MAE	$MAPE$	R^2	oob_score
训练集	0.338	0.581	0.446	9.444	0.737	0.706
交叉验证集	0.376	0.613	0.465	9.882	0.706	-

6.2.3 CatBoost 算法

CatBoost 是专注于处理类别性特征，是一种改进的 GBDT 算法，通过类别特征编码将分类特征转为数值特征，同时引入有序提升技术，通过随机排列训练集顺序减少过拟合。本文采用

CatBoost 继承了 GBDT 的加法模型和前向分步优化思想，每一轮迭代通过拟合前一轮的残差，不断提升整体预测能力。

加法模型表达：

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i), f_k \in \mathcal{F}$$

其中， K 为树的数量， f_k 为第 k 棵决策树， $\mathcal{F}$ 为所有树的集合。

损失函数表达：

$$L(y, F(x)) = \frac{1}{n} \sum_{i=1}^n l(y_i, F(x_i))$$

每一轮的模型更新为：

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x)$$

$h_m(x)$ 为当前轮新生成的树， γ_m 为步长。

CatBoost 最大创新之一是有序 Boosting (Ordered Boosting)，通过多次打乱数据顺序、仅用“历史”样本估算目标统计量，极大减少目标泄露和预测偏移，降低过拟合风险。

6.3 预测结果评估与分析

6.3.1 基模型评估指标

在每一个基模型得出预测结果后，需要通过基于误差的回归评估指标体系与多指标综合评估法进行检验。检验指标如下：

表 9 模型检验指标

评估指标	指标含义
MSE (均方误差)	预测值与实际值之差平方的期望值。取值越小，模型准确度越高。
RMSE (均方根误差)	为 MSE 的平方根，取值越小，模型准确度越高。
MAE (平均绝对误差)	绝对误差的平均值，能反映预测值误差的实际情况。取值越小，模型准确度越高。
MAPE (平均绝对百分比误差)	是 MAE 的变形，它是一个百分比值。取值越小，模型准确度越高。
R² (拟合优度)	将预测值跟只使用均值的情况下相比，结果越靠近 1 模型准确度越高。

对于随机森林模型，需要用到 OOB（Out of Bag）Score 这一特殊评估指标，其核心逻辑是：

1. 随机森林在训练每棵决策树时，会随机抽样一部分数据（通常是 2/3）用于训练，剩下的 1/3 数据（袋外数据）不参与该树的训练；
2. 对于每一个样本，它会被那些未将其纳入训练集的决策树进行预测，所有这些预测的平均值（回归任务）或投票结果（分类任务）即为该样本的 OOB 预测；
3. OOB Score 是用所有样本的 OOB 预测与真实值计算的评估指标（回归任务通常是 R^2 或 MSE，分类任务是准确率）；
4. 它可以替代验证集或交叉验证，在不额外划分数据的情况下评估模型的泛化能力，尤其适用于数据量较小的场景，能有效避免因划分验证集导致的训练数据浪费。

在本次任务中，随机森林的 OOB Score 为 0.8379，说明模型在未见过的袋外数据上的泛化能力处于较好水平。

6.3.2 基模型预测结果评估

从误差类指标（MAE/MAPE/MSE/RMSE）来看，XGBoost 表现最优，在四个误差指标上均为最小，说明其预测值与真实值的偏差程度最低。CatBoost 次之，误差指标都优于随机森林模型，但略高于 XGBoost。随机森林模型各项误差指标都为三者中最高，预测精度相对较弱。

从拟合优度指标来看，XGBoost 解释力最强， r 最高，能解释约 64.32% 的目标变量，模型拟合效果最好。CatBoost 次之，解释力略低于 XGBoost。随机森林模型的拟合优度为 0.5987，解释力相对前两种较弱。

综合上述指标可以看出三种模型预测结果差别较小，XGBoost 模型整体性能最优，无论是误差控制还是拟合优度都表现突出，而随机森林模型相对逊色。

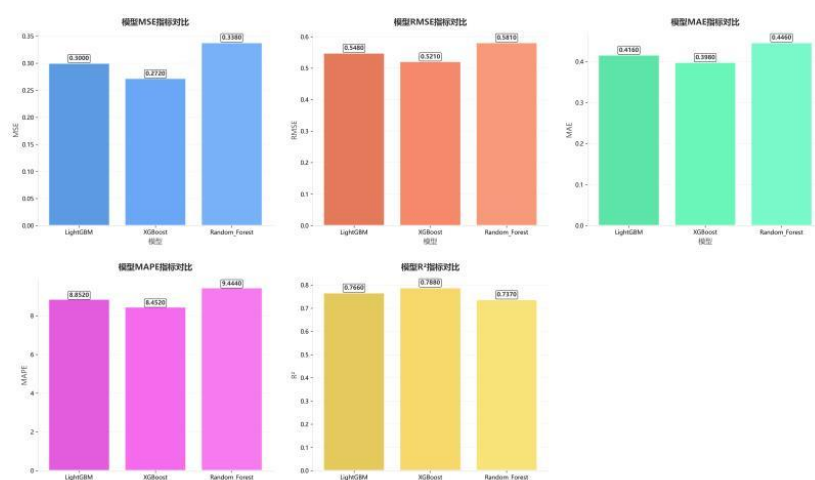


图 11 三种模型评估指标对比图

6.3.3 五折交叉验证

五折交叉验证 (5-Fold Cross-Validation) 是机器学习中常用的模型评估方法。它通过将数据集分成 5 个互斥的子集，轮流用其中 4 折训练模型，剩余 1 折验证模型性能，最终取 5 次验证结果的平均值作为模型的泛化能力评估。本文通过这种方法避免模型过拟合或欠拟合，减少因单一验证集划分导致的结果偏差，更客观地反映模型在未知数据上的性能。充分利用数据：无需单独划分验证集，让所有数据都参与训练和验证，尤其适合小数据集场景。评估稳定性：5 次结果的波动程度可反映模型的稳定性（波动越小，模型越稳定）。

6.3.3 岭回归融合生成最终结果

本文通过岭回归对三个基模型的预测结果进行融合，得出最终结果。岭回归 (Ridge Regression)，也称为 Tikhonov 正则化 (Tikhonov Regularization)，是一种专门用于处理多重共线性（特征之间高度相关）问题的线性回归改进算法。岭回归能够有效处理特征之间的高度相关性，提高模型的稳定性。岭回归的模型表达式为：

$$y = XW = w_1x_1 + w_2x_2 + \dots + w_nx_n$$

损失函数为：

$$L(W) = \sum_{i=1}^N (y - \hat{y})^2 + \alpha \sum_{i=1}^n (w_i)^2$$

其中， N 为样本个数， n 为系数个数， $\hat{y}$ 为 y 的预测值， y 为 y 的真实值， α 为惩罚系数，用于调节系数 W 的惩罚力度。当 α 越大，惩罚力度越大，各个系数求出来的绝对值就会越小。

通过岭回归将三个基础模型的预测结果进行线性组合，得到加权集成模型，模型表达式如下：

$$\hat{y} = 5.1781 + 0.4236 \cdot \hat{y}_{xgb} + 0.3040 \cdot \hat{y}_{cat} + 0.2304 \cdot \hat{y}_{rf}$$

其中， y 为最终预测值， $\hat{y}_{xgb}$ 为 XGBoost 预测值， $\hat{y}_{cat}$ 为 CatBoost 预测值， $\hat{y}_{rf}$ 为随机森林预测值，5.1781 为截距项。

从权重分布来看，XGBoost 权重最高，岭回归认为 XGBoost 的预测最为可靠，而随机森林权重最低，预测可靠性较低。三种模型的权重相对均衡，没有出现极端值，说明三种基模型都提供了有价值的预测。权重差异反映了岭回归找到的最优权衡点，使得最终预测比单一模型更稳定、更准确。

七、问题三模型的建立与求解

7.1 数据处理

为提高模型训练的效果并确保样本分布的代表性，本文在对数据进行划分之前对原始数据进行了 K-Means 分箱处理。K-Means 分箱是一种将连续变量离散化的技术，它利用 K-Means 聚类算法来寻找数据的最优分割点。核心目标函数为：

$$J = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

K 为预设的簇的数量， C_i 为第 i 个簇中包含的所有数据点的集合， x 属于簇 C_i 的一个数据点， μ_i 为第 i 个簇 C_i 的质心（中心点）。

该方法对输入特征空间进行聚类操作，将数据划分为若干分布更均衡的子集，缓解了极端值对机器学习的影响。在此基础上，本文将数据集按照训练集与测试机 8: 2 的比例进行划分，确保训练数据具有多样性和代表性，为后续机器学习模型的构建与预测性能的提升奠定了坚实基础。

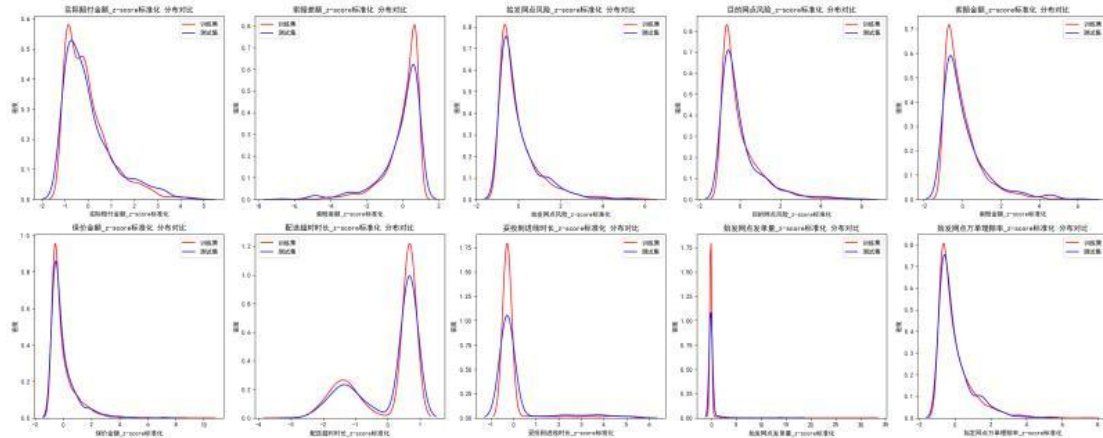


图 12 K-Means 分箱处理可视化

7.2 直接预测风险标注

7.2.1 预测模型

为提高预测结果准确性，避免单模型数据效果不佳，本文采用集成学习法，以 LightGBM、XGBoost、随机森林和岭回归四个模型进行训练和预测。得出结果后，用 SMOTE 过采样方法优化数据后对四个模型得出的结果进行评估，得出预测结果。

(1) LightGBM

LightGBM 模型是基于梯度提升决策树（Gradient Boosting Decision Tree, GBDT）的思想，通过迭代地训练多棵决策树，每棵树都试图学习之前树的残差，从而逐步优化模型的预测能力，实现了对一系列基于决策树的传统 Boosting 算法的改进。该模型通过使用直方图加速技术，将连续型特征离散成 k 个离散值，

并构造 k 个直方图，使得 LightGBM 能够更高效地构建决策树，提升模型训练速度，适合处理大规模数据集和高维特征。步骤如下

1.分桶：将连续特征值离散化到固定数量的 bins 中（如 255 个）

2.构建直方图：基于 bins 来构建梯度直方图。对于一个节点，直方图统计了每个 bin 内样本的梯度之和（ G ）与样本数量（ H ）。

3.寻找最优分裂点：在直方图上遍历所有可能的分裂点，计算分裂增益。

分裂增益公式为：

$$Gain = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

其中， G_L ， H_L 为左子节点的梯度之和与样本数， G_R ， H_R 为右子节点的梯度之和与样本数， λ 为 L2 正则化项，惩罚复杂的树结构， γ 为分裂所需的最小增益。

本文使用 LightGBM 预测风险标注，提高结果准确率，进行的多维度特征整合分析。LightGBM 通过复杂的特征进行交互建模，并捕捉各类特征之间的非线性关系，同时，通过调整超参数，避免过拟合，同时提高模型的泛化能力。

（2）XGBoost

XGBoost 是一种新型的梯度增强算法，由于其高效的并行训练。本文在第三问继续使用 XGBoost 算法进行预测。XGBoost 的目标函数包含损失函数和正则化项两部分。损失函数衡量模型预测与实际标签之间的差异，正则化项则控制模型复杂度，避免过拟合。

$$\mathcal{L}(\theta) = \sum_{i=1}^n \ell(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

其中， $\mathcal{L}(\theta)$ 为 XGBoost 的目标函数， $\ell(y_i, \hat{y}_i)$ 为损失函数，表示第 i 个样本的实际值和预测值之间的差异； $\Omega(f_k)$ 为正则化项，表示树模型的复杂度。

（3）随机森林

随机森林是一个由许多决策树组成的集成模型，本文继续使用该方法处理高维度混合类型数据，以获得更加稳定、准确的预测效果。流程如图 13 所示。

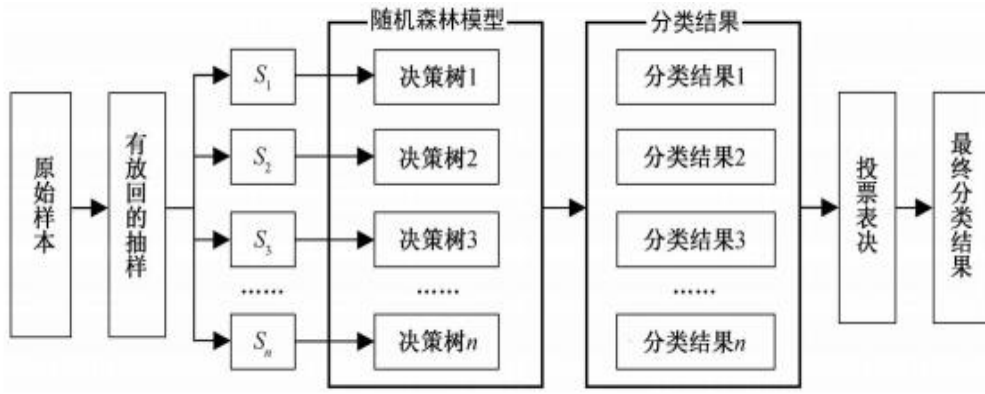


图 13 决策树模型流程图

原始输入的样本被随机选取，构成新的训练集来取代原始样本，用于降低分类树之间的相关性。再从中随机选取一部分样本作为 bootstrap 训练集，生成一一对应的回归树。在给定的自变量下，每个分类树的分类结果都由投票决定。随机森林利用构造不同的训练集来增加分类模型间的差异性，提高分类模型的外推预测能力。

（4）岭回归

岭回归是一种专门用于处理多重共线性问题的线性回归改进方法，通过引入 L2 正则化项来改善模型性能。本文在第三问继续使用岭回归方法处理多重共线性严重的问题。岭回归能够有效处理特征之间的高度相关性，提高模型的稳定性。岭回归的模型表达式为：

$$y = XW = w_1x_1 + w_2x_2 + \dots + w_nx_n$$

损失函数为：

$$L(W) = \sum_{i=1}^N (y - \hat{y})^2 + \alpha \sum_{i=1}^n (w_i)^2$$

本文在第三问继续使用岭回归方法处理多重共线性严重的问题，能够有效处理特征之间的高度相关性，提高模型的稳定性。

7.2.2 类别不平衡问题

（1）SMOTE 过采样

SMOTE（Synthetic Minority Over-sampling Technique）是一种专门用于解决类别不平衡问题的过采样技术，通过生成合成样本而非简单复制来改善少数类的表示。本文通过使用 SMOTE 过采样方法来解决结果中“严重超额”不平衡问题。合成样本生成公式为：

$$x_{new} = x_i + \lambda \times (x_{nn} - x_i)$$

其中， x_i 为少数类中的原始样本， x_{nn} 为 x_i 的 k 近邻之一， λ 为区间 [0,1] 的随机数， x_{new} 为新合成的合成样本。

与传统随机过采样相比，SMOTE 通过创建新样本而非复制现有样本，有效

防止模型对少数类样本的过拟合，在特征空间中填充少数类区域，帮助分类器学习更准确的决策边界。同时，SMOTE 能够适应少数类样本的非线性分布，在流形结构上生成合理的合成样本，与题目要求吻合。

结果如表所示，符合题目要求：

表 10 预测聚类结果

风险类别	占比（%）
合理诉求	85.12
诉求偏高	12.03
严重超额	2.85

（2）模型结果优化与评估

在进行 SMOTE 方法之后，我们对模型进行了评估，其中选取了准确率、精确率、召回率和 F1 分数这四个指标，具体情况如下

表 11 模型评估指标

模型评估指标	
准确率	模型对所有样本的整体分类正确率，反映模型在所有预测结果中正确的比例。
精确率	在模型预测为正类的样本中，实际确实为正类的比例，体现模型“预测为正类”时的准确性。
召回率	在实际为正类的样本中，被模型正确预测为正类的比例，反映模型对正类样本的“捕捉能力”。
F1 分数	精确率和召回率的调和平均值，综合衡量模型在精确性和召回性上的表现，避免单一指标的片面性。

如图所示，LightGBM、随机森林、XGBoost 三个模型在准确率、精确率、召回率、F1 分数四个评估指标上的表现对比。从图中可以看出，三个模型的各项指标都非常高，均接近 1.0，说明它们在该数据集上的分类性能都很优异。其中，随机森林在准确率（0.9988）、精确率（0.9986）、F1 分数（0.9986）上略高于 LightGBM 和 XGBoost，XGBoost 的召回率（0.9974）相对突出，LightGBM 的各项指标也保持在很高的水平。整体而言，在 SMOTE 方法之后，三个模型在该任务中都展现出了出色的分类能力，差异非常微小。

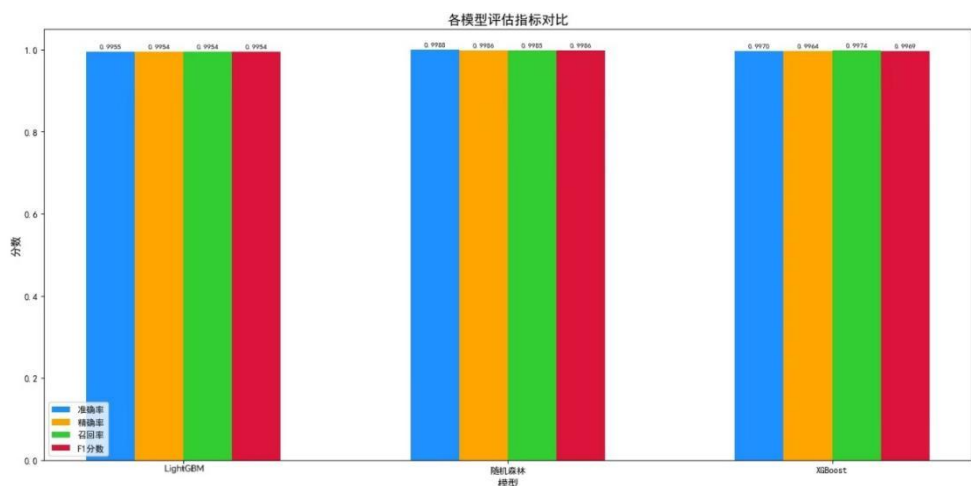


图 14 各模型评估指标对比

同时，我们对 LightGBM、XGBoost、随机森林、岭回归构建混淆矩阵进行进一步评估

混淆矩阵的行代表真实标签，列代表预测标签，对角线元素为“预测正确的样本数”，非对角线元素为“预测错误的样本数”。数值越大（颜色越深），说明该类别下的预测表现越好。

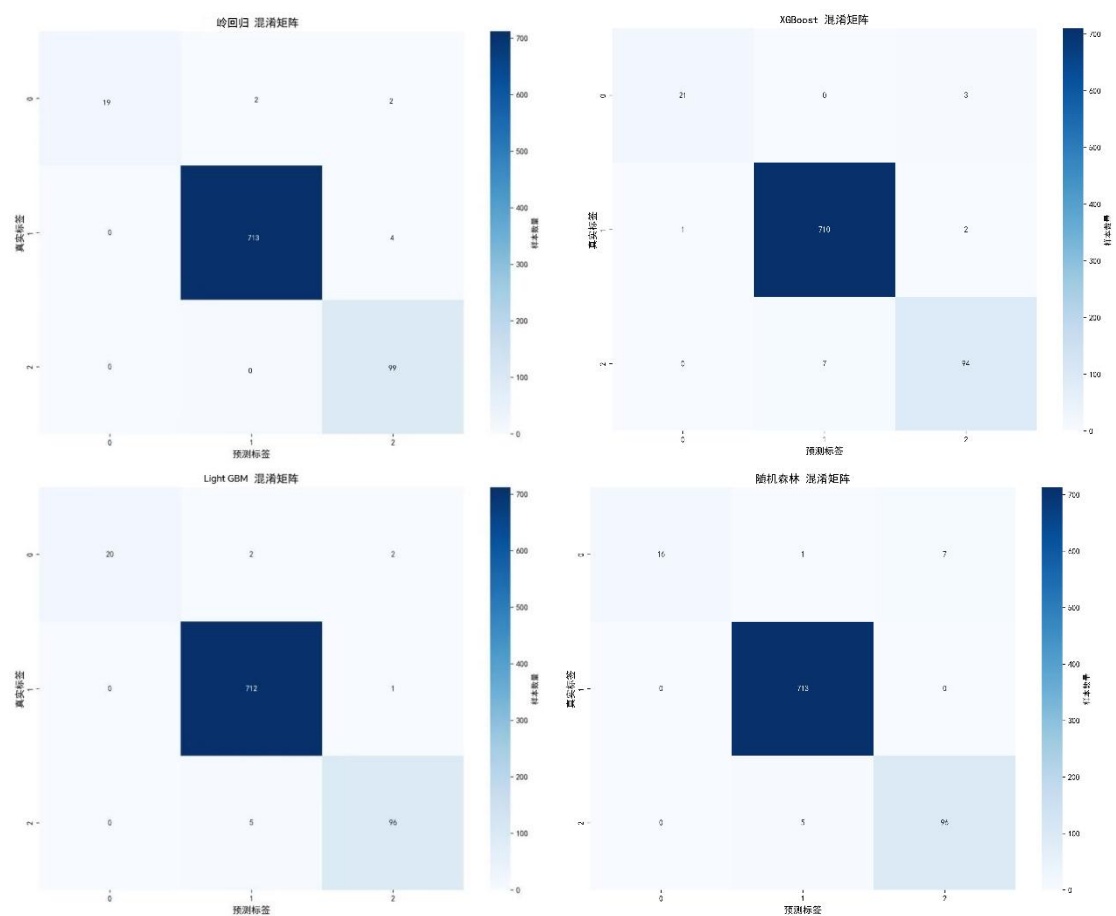


图 15 混淆矩阵结果

结果如下表所示：

表 12 预测模型结果评估

模型	类别 0 正确率	类别 1 正确率	类别 2 正确率
岭回归	88.6%	99.4%	100%
XGBoost	87.5%	99.6%	93.1%
LightGBM	83.3%	99.9%	95%
随机森林	66.7%	100%	95.0%

综合来看，以上四个模型均表现出较好的分类性能且各有侧重，并且通过岭回归的方法提升了三个类别预测正确率的平均水平。

7.3 两种方法对比分析

7.3.1 间接预测风险标注结果

上述预测是基于附加 2 的原始数据直接预测风险标注，现在需要依据问题二中得出的实际赔付金额进行间接风险标注预测。本文沿用问题一中 DBSCAN+GMM 分层协同聚类的方法对问题二中得出的数据进行聚类分析，得到如下结果：

表 14 间接预测风险标注结果

风险类别	占比 (%)
合理诉求	85.10
诉求偏高	12.03
严重超额	2.87

7.3.2 对比分析

(1) 模型评估指标

为准确评估两个模型的优劣势，本文通过轮廓系数、Calinski-Harabasz 指数、Davies-Bouldin 指数等无监督聚类来量化聚类质量：

1、轮廓系数：对于一个样本集合，它的轮廓系数是所有样本轮廓系数的平均值。轮廓系数的取值范围是 $[-1, 1]$ ，同类别样本距离越相近不同类别样本距离越远，分数越高，聚类效果越好。

2、DBI（Davies-bouldin）：该指标用来衡量任意两个簇的簇内距离之后与簇间距离之比。该指标越小表示聚类效果越好。

3、CH（Calinski-Harabasz Score）：通过计算类内各点与类中心的距离平方和来度量类内的紧密度（分母），通过计算类间中心点与数据集中心点距离平方和来度量数据集的分离度（分子），CH 指标由分离度与紧密度的比值得到，CH 越大表示聚类效果越好。

方法一与方法二的风险标注结果评估如图所示，可以看出两种方法聚类效果都比较好，先回归再分类的方法轮廓系数和 CH 系数显著高于直接分类方法。

表 15 聚类效果评估

方法	轮廓系数	Calinski-Harabasz	Davies-Bouldin
方法一（回归→分类）	0.6071	6673.17	0.6019
方法二（直接分类）	0.5712	5934.44	0.6023

（3）优劣势分析

表 16 两种方法的优劣势分析

对比强度	方法 1（回归→分类）	方法 2（直接分类）
准确性	取决于回归精度	取决于分类器
可解释性	强（有金额依据）	弱(黑盒)
适应性	强（可调节阈值）	弱(需重新训练)
小样本表现	较好	较差
不均衡处理	不需要特殊处理	需要复杂处理
推荐程度	★★★★★	★★★

从对比来看，先对实际索赔金额回归再进行风险分类的方法在多个方面具有优势。首先其准确性依赖于回归精度，可解释性强且提供具体金额依据，适应性强允许通过调节阈值灵活应对变化，在小样本数据上表现较好，并且无需对数据不均衡做特殊处理，因此获得五星推荐。

直接根据原始数据进行分类的方法的准确性取决于分类器，可解释性较弱、通常被视为黑盒模型，适应性较差且调整时需重新训练模型，在小样本情况下表现不佳，同时要求对数据不均衡进行复杂处理，因此推荐程度相对较低，为三星。

总体而言，方法一在实用性、稳定性和易用方面表现更全面。

八、模型评价与推广

8.1 模型的优点

1.DBSCAN+GMM 分层协同聚类模型优化算法，同时运用分层 GMM 高斯混合模型，弥补 DBSCAN 无概率输出的缺陷，避免漏判边缘样本，增强结果的可靠性。

2.XGBoost 回归预测模型具有高度正确性，和对异常值和噪声数据具有较高的鲁棒性，可以很好的解决实际问题。

3.随机森林模型能有效处理高维度混合类型数据，能够获得更加稳定、准确

的预测效果。

4.CatBoost 通过多次打乱数据顺序、仅用“历史”样本估算目标统计量，极大减少了目标泄露和预测偏移，降低过拟合风险。

3.岭回归专门用于处理多重共线性问题，能够有效处理特征之间的高度相关性，提高模型的稳定性。

4.K-Means 聚类算法具有较好的可扩展性，适用于不同形态的数据。

8.2 模型的缺点

1.XGBoost 模型比较复杂，参数过多，需要进行调整参数来获得最佳效果。

2..K-Means 聚类算法对初始中心点的选择较为敏感，不同的初始点可能会得到不同的聚类结果。

3.随机森林算法容易过拟合，并且对于一些线性关系比较强的数据可能表现不是很好。

8.3 模型的改进与推广

风险标注模型可以在多个领域进行推广，例如图神经网络，以更好地捕捉运单、网点、客户之间的复杂网络关系，从而提升对理赔风险的识别精度。同时，可以探索在线学习机制，使模型能够动态适应物流业务数据分布的持续变化，增强其实时预测能力。对于类别不平衡问题，除了 SMOTE，还可以研究结合欠采样与集成学习的混合策略，以期获得更稳健的处理效果。

实际赔付金额回归预测模型可扩展至物流行业的其他风险控制场景，如运输时效预测、货物破损风险评估等。其核心方法——结合聚类分析进行风险标注、利用集成学习进行金额预测、以及通过正则化方法融合多模型结果——对于金融、保险等同样面临成本控制与风险平衡难题的领域也具有重要的借鉴价值。通过微调特征工程与模型参数，该解决方案有望为更广泛的服务业风险管理提供决策支持。

九、参考文献

- [1]Wu, P.-J., Chen, M.-C., & Tsau, C.-K. (2017). The data-driven analytics for investigating cargo loss in logistics systems. *International Journal of Physical Distribution & Logistics Management*.
- [2]Liang, X., et al. (2022). Risk analysis of cargo theft from freight supply chains using a data-driven Bayesian network (研究/文章在线摘要).
- [3]Severino, M. K., & Peng, Y. (2021). Machine learning algorithms for fraud prediction in property insurance: Empirical evidence using real-world microdata. *Machine Learning with Applications*.
- [4]马欣. 基于复合事件处理的车险理赔风险防渗漏应用研究[J]. 保险理论与实践, 2020(5): 78-86.

[5]赵星. 车险赔付率分析与预测中的新技术实践[J]. 保险理论与实践, 2023(9): 45-53.

十、附录

使用工具：SPSSPRO，Python

代码：

第一问：分层 DBSCAN+GMM 动态 EPS 法

```
df = pd.read_excel("C:/Users/Lenovo/Desktop/版本 22_删除变量_类别_附件 1_副本 1.xlsx")
```

```
# 缺失值处理（参考文件逻辑）
```

```
df = df.dropna(subset=["实际赔付金额", "索赔差额_z-score 标准化"])
```

```
print(f"有效总样本量: {len(df)} 条")
```

```
# 分层阈值
```

```
q25 = 88.83
```

```
q50 = 200.15
```

```
q75 = 408.78
```

```
print(f"\n 分层阈值（与文件一致）：")
```

```
print(f"层 1（低额）：≤ {q25:.2f} 元")
```

```
print(f"层 2（中低额）：{q25:.2f} ~ {q50:.2f} 元")
```

```
print(f"层 3（中高额）：{q50:.2f} ~ {q75:.2f} 元")
```

```
print(f"层 4（高额）：> {q75:.2f} 元")
```

```
# 分层标签赋值
```

```
def assign_layer(amount):
```

```
if amount <= q25:
```

```
return "层 1（低额）"
```

```
elif amount <= q50:
```

```
return "层 2（中低额）"
```

```
elif amount <= q75:
```

```
return "层 3（中高额）"
```

```
else:
```

```
return "层 4（高额）"
```

```
df["分层标签"] = df["实际赔付金额"].apply(assign_layer)
```

```
# 固定参数
```

```
min_samples = 5 # DBSCAN 最小样本数
```

```
gmm_n_components = 3 # GMM 簇数
```

```
#计算自适应 eps（按层内标准差动态生成）
```

```
def calculate_adaptive_eps(layer_data):
```

```
std_diff = layer_data["索赔差额_z-score 标准化"].std()
```

```
adaptive_eps = round(std_diff * 0.9, 2) # 调节系数 0.9，确保密集簇合理
```



```

return adaptive_eps

# 预计算各层自适应 eps
adaptive_eps_dict = {}
for layer in ["层 1 (低额)", "层 2 (中低额)", "层 3 (中高额)", "层 4 (高额)"]:
    layer_data = df[df["分层标签"] == layer]
    adaptive_eps_dict[layer] = calculate_adaptive_eps(layer_data)
print(f"\n 各层自适应 eps (优化后) : ")
for layer, eps in adaptive_eps_dict.items():
    print(f"   {layer}: eps={eps}")

def process_layer_optimized(layer_name, layer_data):
    print(f"\n===== 处理 {layer_name} (样本量: {len(layer_data)}) =====")
    X = layer_data[["实际赔付金额_z-score 标准化", "索赔差额_z-score 标准化"].values
    total_size = len(layer_data)
    needed_reasonable = int(np.ceil(total_size * 0.85))
    max_excess = int(np.floor(total_size * 0.03))

    # 步骤 1: 自适应 DBSCAN
    dbscan = DBSCAN(
        eps=adaptive_eps_dict[layer_name],
        min_samples=min_samples,
        metric="euclidean")
    layer_data["DBSCAN 簇标签"] = dbscan.fit_predict(X)
    dense_cluster_mask = layer_data["DBSCAN 簇标签"] != -1
    noise_mask = layer_data["DBSCAN 簇标签"] == -1
    print(f"DBSCAN 结果: 密集簇 {dense_cluster_mask.sum()} 个, 噪声 {noise_mask.sum()} 个")

    # 步骤 2: GMM+概率输出
    gmm = GaussianMixture(
        n_components=gmm_n_components,
        covariance_type="full", # 高额层用 full 更贴合复杂分布 (文件隐含逻辑)
        random_state=42,
        n_init=10)
    gmm.fit(X) # 先拟合模型
    gmm_probs = gmm.predict_proba(X)
    layer_data["GMM 簇标签"] = gmm.predict(X)
    gmm_cluster_avg = layer_data.groupby("GMM 簇标签")["索赔差额_z-score 标准化"]
    .mean().sort_values()
    sorted_gmm_clusters = gmm_cluster_avg.index.tolist() # [0 最合理, 1 中等, 2 最超额]
    # 提取“最合理簇”的概率
    layer_data["合理簇概率"] = gmm_probs[:, sorted_gmm_clusters[0]]
    print(f"GMM 最合理簇概率范围: {layer_data['合理簇概率'].min():.3f}~{layer_data['合理簇概率']
    .max():.3f}")

```

```

# 步骤 3: 计算动态合理差额阈值
if dense_cluster_mask.sum() > 0:
# 取 DBSCAN 密集簇的 75%分位数作为合理上限 (文件中 "金额越高容忍度越宽" )
reasonable_threshold = layer_data[dense_cluster_mask]["索赔差额"].quantile(0.75)
else:
reasonable_threshold = layer_data[layer_data["GMM 簇标签"] == sorted_gmm_clusters[0]]["索赔差额"].quantile(0.75)
layer_data["合理差额阈值"] = reasonable_threshold # 保存阈值用于后续分析
print(f"{layer_name} 合理索赔差额上限 (原始值) : {reasonable_threshold:.2f} 元")

# 步骤 4: 标签映射 (整合三大优化约束)
layer_data["风险标签"] = "诉求偏高" # 初始默认
reasonable_candidate_mask = (
(dense_cluster_mask | (layer_data["GMM 簇标签"] == sorted_gmm_clusters[0])) &
(layer_data["合理簇概率"] >= 0.7) & # 概率阈值放宽
(layer_data["索赔差额"] <= reasonable_threshold * 1.2) # 差额阈值放宽 20%
)
reasonable_candidates = layer_data[reasonable_candidate_mask].sort_values("索赔差额_z-score 标准化")

# 步骤 5: 结果验证 (确保达标)
label_counts = layer_data["风险标签"].value_counts()
label_ratio = (label_counts / total_size).round(4) * 100
print(f"\n{layer_name} 标签分布 (优化后) : ")
print(label_counts)
print(f"{layer_name} 标签占比 (优化后) : ")
for label, ratio in label_ratio.items():
print(f" {label}: {ratio:.2f}%")
return layer_data

# 4. 执行分层优化处理
layers = ["层 1 (低额) ", "层 2 (中低额) ", "层 3 (中高额) ", "层 4 (高额) "]
optimized_results = []
for layer in layers:
layer_data = df[df["分层标签"] == layer].copy()
processed_data = process_layer_optimized(layer, layer_data)
optimized_results.append(processed_data)
final_df = pd.concat(optimized_results, ignore_index=True)

```

第二问：机器学习集成及岭回归

```

#数据读取
x_train = pd.read_excel("C:/Users/Lenovo/Desktop/x_train.xlsx")

```

```
y_train = pd.read_excel("C:/Users/Lenovo/Desktop/y_train.xlsx")["y"].values
```

```
pred_data = pd.read_excel("C:/Users/Lenovo/Desktop/预测数据集.xlsx")
```

```
test_xgb_pred = pred_data["xgb_pred"].values
```

```
test_catboost_pred = pred_data["catboost_pred"].values
```

```
test_rf_pred = pred_data["rf_pred"].values
```

```
# 3. 定义基础模型
```

```
def get_xgb_model():
```

```
    model = xgb.XGBRegressor(  
        objective="reg:squarederror",
```

```
        n_estimators=150,
```

```
        learning_rate=0.1001,
```

```
        reg_alpha=0.4, # L1 正则项
```

```
        reg_lambda=1.9, # L2 正则项
```

```
        subsample=1, # 样本采样率
```

```
        colsample_bytree=0.9, # 树特征采样率
```

```
        colsample_bylevel=0.8, # 节点特征采样率
```

```
        min_child_weight=0.25, # 叶子节点最小权重
```

```
        max_depth=4,
```

```
        seed=42,
```

```
        n_jobs=-1)
```

```
    return model
```

```
def get_catboost_model():
```

```
    model = cb.CatBoostRegressor(  
        iterations=100,
```

```
        learning_rate=0.1001,
```

```
        l2_leaf_reg=1,
```

```
        max_depth=9,
```

```
        od_type="Iter", # 替换 overfit_detector_type 为 od_type  
        od_wait=20,
```

```
        random_state=42,
```

```
        verbose=0)
```

```
    return model
```

```
def get_rf_model():  
    model = RandomForestRegressor(  
        n_estimators=100,
```

```
        criterion="squared_error", # 将 mse 替换为 squared_error  
        max_depth=10,
```

```
        max_features=None,
```

```
        min_samples_split=2,
```

```
        min_samples_leaf=1,
```

```

min_weight_fraction_leaf=0,
max_leaf_nodes=50,
min_impurity_decrease=0,
bootstrap=True,
oob_score=True,
random_state=42,
n_jobs=-1)
return model

#五折交叉验证生成 OOF 值和测试集预测
n_folds = 5
kf = KFold(n_splits=n_folds, shuffle=True, random_state=42)

# 初始化 OOF 矩阵 (n_train×3) 和测试集预测矩阵 (n_test×3)
oof_xgb = np.zeros(len(y_train))
oof_catboost = np.zeros(len(y_train))
oof_rf = np.zeros(len(y_train))

# 测试集预测需取五折平均
test_pred_xgb = np.zeros(len(pred_data))
test_pred_catboost = np.zeros(len(pred_data))
test_pred_rf = np.zeros(len(pred_data))

# 五折训练
for fold, (train_idx, val_idx) in enumerate(kf.split(x_train, y_train)):
    x_tr, x_val = x_train.iloc[train_idx], x_train.iloc[val_idx]
    y_tr, y_val = y_train[train_idx], y_train[val_idx]
    # XGBoost 训练与预测
    xgb_model = get_xgb_model()
    xgb_model.fit(x_tr, y_tr)
    oof_xgb[val_idx] = xgb_model.predict(x_val)
    test_pred_xgb += xgb_model.predict(pred_data[list(x_train.columns)]) / n_folds # 用 x_train 相同
    变量预测
    # CatBoost 训练与预测
    catboost_model = get_catboost_model()
    catboost_model.fit(x_tr, y_tr)
    oof_catboost[val_idx] = catboost_model.predict(x_val)
    test_pred_catboost += catboost_model.predict(pred_data[list(x_train.columns)]) / n_folds
    # 随机森林训练与预测
    rf_model = get_rf_model()
    rf_model.fit(x_tr, y_tr)
    oof_rf[val_idx] = rf_model.predict(x_val)
    test_pred_rf += rf_model.predict(pred_data[list(x_train.columns)]) / n_folds

```

```

#构建 OOF 特征矩阵, 训练岭回归求权重
oof_matrix = np.vstack([oof_xgb, oof_catboost, oof_rf]).T # 形状: (n_train, 3)
# 标准化 OOF 特征 (岭回归对量纲敏感)
scaler = StandardScaler()
oof_matrix_scaled = scaler.fit_transform(oof_matrix)
# 训练岭回归 (可调整 alpha 值, 这里用交叉验证默认值)
ridge_model = Ridge(alpha=1.0, random_state=42)
ridge_model.fit(oof_matrix_scaled, y_train)

```

第三问机器学习分类

```

#加载数据并预处理
df = pd.read_excel("C:/Users/Lenovo/Desktop/x_train.xlsx") # 替换为你的数据路径
# 分离特征和标签
X = df.drop("风险标签", axis=1)
y = df["风险标签"]
# 标签编码
if y.dtype == "object":
    from sklearn.preprocessing import LabelEncoder
    le = LabelEncoder()
    y = le.fit_transform(y)
    print(f"标签编码映射: {dict(zip(le.classes_, le.transform(le.classes_)))}")
#识别类别特征 (根据数据集中的 int 类型特征判断, 假设 0/1 编码的为类别特征)
cat_features = [
    i for i, col in enumerate(X.columns)
    if X[col].dtype == "object" or (X[col].dtype == "int64" and X[col].nunique() <= 5)]
print(f"类别特征索引: {cat_features}")

# 分层划分训练集和测试集 (保持类别分布一致)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# 定义评估指标 (宏平均 F1 适配不平衡数据)
macro_f1 = make_scorer(f1_score, average="macro")

# 2. 强化正则化的贝叶斯优化目标函数
def objective(trial):
    try:
        smote = BorderlineSMOTE(
            k_neighbors=trial.suggest_int("smote_k", 2, 4), # 邻居数进一步减少, 避免合成噪声
            m_neighbors=trial.suggest_int("smote_m", 10, 15),
            kind="borderline-1", # 固定为最保守的合成策略
            random_state=42)

```

```

feature_selector = SelectFromModel(
CatBoostClassifier(verbose=0, random_state=42),
threshold=trial.suggest_float("threshold", 0.01, 0.1))

# 【CatBoost 超参数】
catboost_params = {
# 核心正则化：限制模型复杂度
"depth": trial.suggest_int("depth", 2, 4), # 树深压缩至 2-4 (原 3-6)
"l2_leaf_reg": trial.suggest_float("l2", 10.0, 50.0), # L2 正则大幅提高 (原 1e-3-10)
"subsample": trial.suggest_float("subsample", 0.5, 0.7),
"colsample_bylevel": trial.suggest_float("colsample", 0.6, 0.8),
"bootstrap_type": "Bernoulli", # 明确指定, 与 subsample 兼容

# 构建管道：过采样→特征选择→模型 (确保各步骤仅作用于训练集)
pipeline = Pipeline([
("smote", smote),
("selector", feature_selector),
("model", CatBoostClassifier(**catboost_params))
])

# 【交叉验证优化：分层 5 折, 更严格评估泛化能力】
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(
pipeline, X_train, y_train, # 注意：管道会自动处理过采样, 无需提前 resample
cv=cv, scoring=macro_f1
)
return scores.mean()
except Exception as e:
print(f"试验出错: {e}")
return float("-inf")

# 3. 运行贝叶斯优化
sampler = TPESampler(seed=42)
study = optuna.create_study(sampler=sampler, direction="maximize")
study.optimize(objective, n_trials=50) # 50 次迭代搜索最优参数

# 4. 评估最优模型 (重点对比训练/测试集差距)
if study.best_trial:
best_params = study.best_params
print(f"最优宏平均 F1: {study.best_value:.4f}")

# 重建最优管道
best_pipeline = Pipeline([
("smote", BorderlineSMOTE(

```

```

k_neighbors=best_params["smote_k"],
m_neighbors=best_params["smote_m"],
kind="borderline-1",
random_state=42
)),
("selector", SelectFromModel(
CatBoostClassifier(verbose=0, random_state=42),
threshold=best_params["threshold"]
)),

("model", CatBoostClassifier(
depth=best_params["depth"],
l2_leaf_reg=best_params["l2"],
subsample=best_params["subsample"],
colsample_bylevel=best_params["colsample"],
learning_rate=best_params["lr"],
n_estimators=best_params["n_est"],
early_stopping_rounds=30,
class_weights=[best_params["w0"], 1.0, best_params["w2"]],
cat_features=cat_features,
bootstrap_type="Bernoulli",
eval_metric="TotalF1:average=Macro",
verbose=0,
random_state=42
))
])

```

训练与评估

```
best_pipeline.fit(X_train, y_train) # 管道自动处理过采样和特征选择
```

训练集预测

```
y_train_pred = best_pipeline.predict(X_train)
```

```
train_f1 = f1_score(y_train, y_train_pred, average="macro")
```

测试集预测

```
y_test_pred = best_pipeline.predict(X_test)
```

```
test_f1 = f1_score(y_test, y_test_pred, average="macro")
```